

# Laboratorio de redes industriales en Docker

Modbus TCP / RTU · OPC UA · MQTT · SCADA · análisis de tráfico

Integración con hardware real: **Controllino MAXI/MEGA** y **ESP32 PLC 14 (Industrial Shields)**

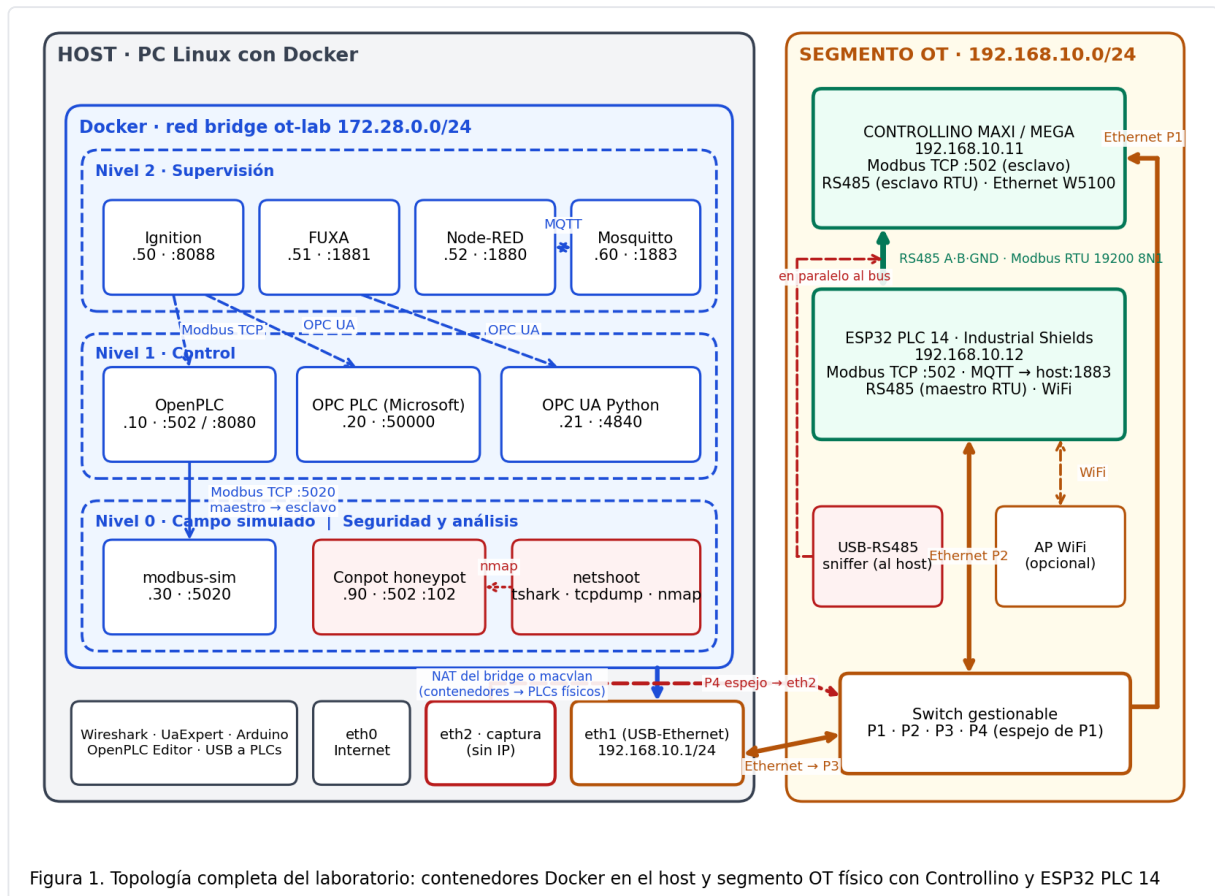


Figura 1. Topología completa del laboratorio: contenedores Docker en el host y segmento OT físico con Controllino y ESP32 PLC 14

# Contenido

---

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>0. Requisitos</b>                                                        | <b>3</b>  |
| <b>1. Arquitectura del laboratorio</b>                                      | <b>4</b>  |
| <b>2. Paso 1 — Estructura del proyecto</b>                                  | <b>5</b>  |
| <b>3. Paso 2 — Modbus TCP</b>                                               | <b>6</b>  |
| 3.1 Esclavo simulado: oitc/modbus-server                                    | 6         |
| 3.2 PLC virtual: OpenPLC                                                    | 6         |
| 3.3 Cliente Modbus desde un contenedor (mbpoll / pymodbus)                  | 7         |
| <b>4. Paso 3 — OPC UA</b>                                                   | <b>8</b>  |
| 4.1 Servidor simulado: OPC PLC de Microsoft                                 | 8         |
| 4.2 Servidor OPC UA propio en Python (asyncua)                              | 8         |
| 4.3 Cliente: UaExpert (host)                                                | 9         |
| <b>5. Paso 4 — SCADA</b>                                                    | <b>10</b> |
| 5.1 Ignition (Inductive Automation)                                         | 10        |
| 5.2 FUXA (open source, ligero)                                              | 11        |
| 5.3 Node-RED (pasarela / pruebas rápidas)                                   | 11        |
| <b>6. Paso 5 — Captura y análisis de tráfico</b>                            | <b>12</b> |
| 6.1 Capturar dentro de la red Docker con netshoot                           | 12        |
| 6.2 Capturar desde el host (Linux)                                          | 12        |
| 6.3 Filtros útiles en Wireshark                                             | 13        |
| 6.4 Escaneo y reconocimiento (contra tu propio lab)                         | 13        |
| <b>7. Paso 6 — Honeypot ICS: Conpot</b>                                     | <b>14</b> |
| <b>8. Paso 7 — Integrar los PLCs físicos: Controllino y ESP32 PLC 14</b>    | <b>15</b> |
| 8.1 Qué tienes en cada equipo                                               | 15        |
| 8.2 Topología física y direccionamiento                                     | 16        |
| 8.3 Cómo llegan los contenedores a los PLCs                                 | 16        |
| 8.4 Firmware: opción rápida con OpenPLC Editor                              | 17        |
| 8.5 Firmware: opción con Arduino IDE (control total)                        | 17        |
| 8.6 Modbus RTU sobre RS485 entre los dos PLCs (y el ESP32 como pasarela)    | 19        |
| 8.7 MQTT desde el ESP32 (IIoT, el otro protocolo que verás en planta)       | 19        |
| <b>9. Lo que no cabe bien en Docker: Ethernet industrial de tiempo real</b> | <b>20</b> |
| <b>10. Ejercicios de red sugeridos (en orden)</b>                           | <b>21</b> |
| <b>11. Problemas frecuentes</b>                                             | <b>22</b> |
| <b>12. docker-compose.yml completo</b>                                      | <b>24</b> |

Casi todo lo de esta guía corre en un solo PC con Docker. Al final tendrás una red simulada con dispositivos de campo, un PLC virtual, servidores OPC UA, un SCADA y un honeypot, **integrada con dos PLCs físicos (Controllino y ESP32 PLC 14 de Industrial Shields)**, y sabrás capturar y analizar el tráfico entre todos ellos.

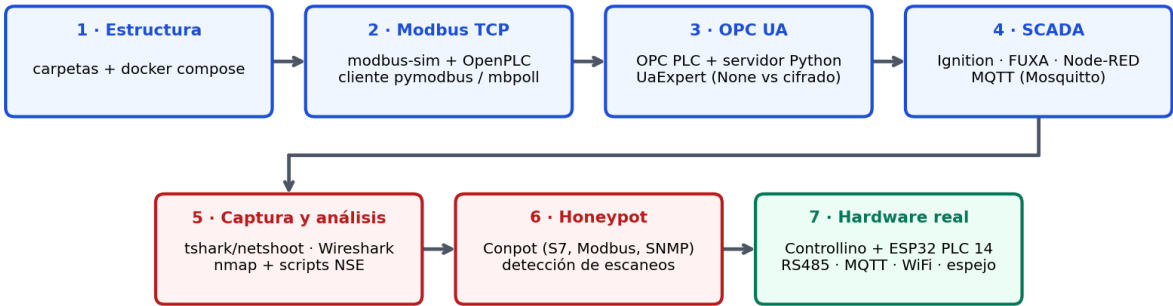


Figura 6. Orden de montaje del laboratorio: cada paso añade una capa sobre la anterior

## 0. Requisitos

| Herramienta                                                                                   | Dónde corre               | Para qué                                                     |
|-----------------------------------------------------------------------------------------------|---------------------------|--------------------------------------------------------------|
| Docker Engine + Docker Compose v2                                                             | Host                      | Toda la infraestructura                                      |
| Wireshark                                                                                     | Host (instalación nativa) | Analizar capturas                                            |
| UaExpert (Unified Automation, gratuito con registro)                                          | Host                      | Cliente OPC UA gráfico                                       |
| Python 3.10+ con pymodbus y asyncua                                                           | Host (opcional)           | Scripts propios                                              |
| Arduino IDE 1.8.x + paquete <code>industrialshields-esp32</code> + boards Controllino         | Host                      | Firmware de los PLCs físicos                                 |
| OpenPLC Editor                                                                                | Host                      | Programar los PLCs en IEC 61131-3 y subirles firmware Modbus |
| Switch Ethernet (idealmente gestionable con <b>port mirroring</b> , p. ej. TP-Link TL-SG105E) | Físico                    | Segmento OT real y captura de tráfico entre PLCs             |
| Adaptador USB-Ethernet (segunda NIC) y adaptador USB-RS485                                    | Físico                    | Segmento OT aislado y análisis de Modbus RTU                 |
| Fuente 24 Vdc ≥ 2 A                                                                           | Físico                    | Alimentar ambos PLCs (no los alimentes solo por USB)         |

Recomendación: si estás en Windows o macOS, usa una **VM Linux (Ubuntu) con Docker** en lugar de Docker Desktop. En Linux ves las interfaces `docker0/br-xxxx` desde Wireshark y puedes usar `network_mode: host`; en Windows/macOS el bridge de Docker vive dentro de una VM oculta y capturar es más incómodo.

# 1. Arquitectura del laboratorio

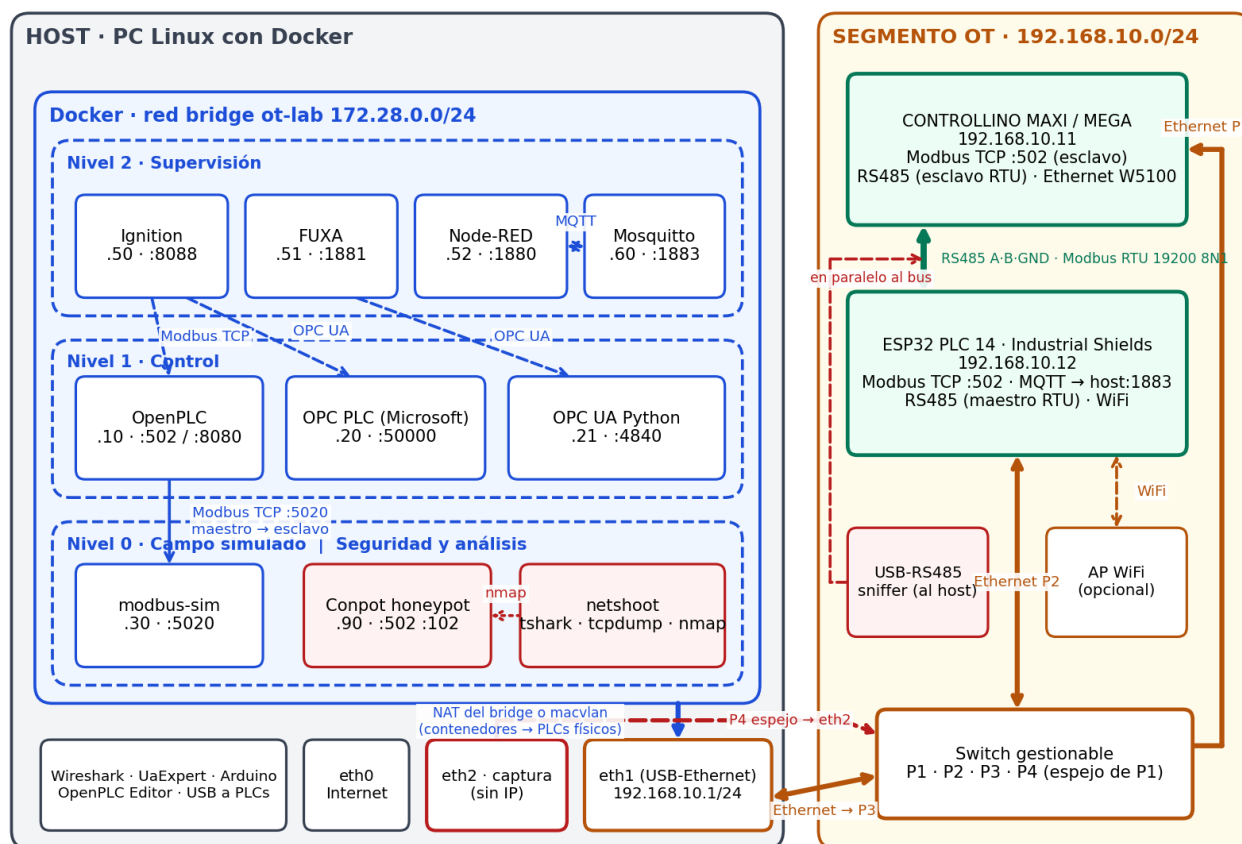


Figura 1. Topología completa del laboratorio: contenedores Docker en el host y segmento OT físico con Controllino y ESP32 PLC 14

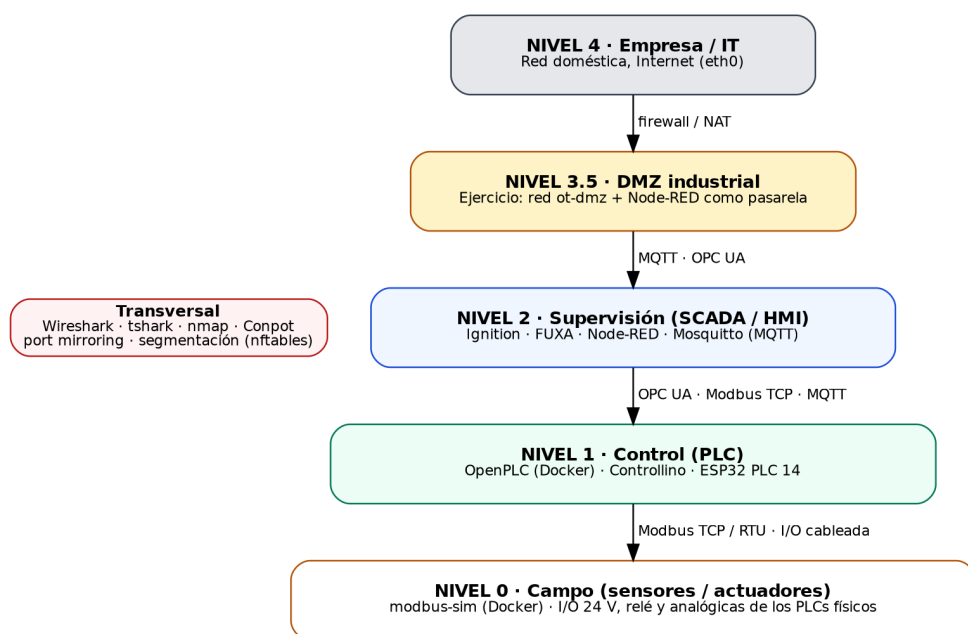


Figura 2. Diagrama de bloques: niveles de la pirámide de automatización y herramienta del laboratorio en cada uno

Usamos IPs fijas para que las capturas y los ejercicios de nmap sean legibles. El segmento físico 192.168.10.0/24 va en una NIC dedicada del host, separado de tu red doméstica: eso ya es un ejercicio de segmentación.

## 2. Paso 1 — Estructura del proyecto

---

```
mkdir -p ~/ot-lab/{modbus,opcua,mosquitto,ignition-data,fuxa-data,nodered-data,captures}  
cd ~/ot-lab
```

Crea el archivo `docker-compose.yml` (está completo al final de esta guía y también como archivo aparte). Los pasos siguientes explican cada servicio; puedes levantarlos uno a uno con `docker compose up -d <servicio>` o todo de golpe con `docker compose up -d`.

## 3. Paso 2 — Modbus TCP

### 3.1 Esclavo simulado: `oitic/modbus-server`

Es un servidor Modbus TCP escrito en Python que permite predefinir registros desde un JSON. Crea `modbus/server.json`:

```
{
  "server": { "listenerAddress": "0.0.0.0", "listenerPort": 5020, "tlsParams": { "flag": false } },
  "registers": {
    "zeroMode": false,
    "initializeUndefinedRegisters": true,
    "discreteInput": { "1": true, "2": false },
    "coils": { "1": false, "2": true },
    "holdingRegister": { "40001": 1234, "40002": 25, "40003": 72 },
    "inputRegister": { "30001": 999 }
  }
}
```

Servicio en compose:

```
modbus-sim:
  image: oitic/modbus-server:latest
  container_name: modbus-sim
  command: -f /server_config.json
  volumes:
    - ./modbus/server.json:/server_config.json:ro
  ports:
    - "5020:5020"
  networks:
    ot-lab: { ipv4_address: 172.28.0.30 }
```

```
docker compose up -d modbus-sim
docker logs modbus-sim
```

### 3.2 PLC virtual: OpenPLC

```
openplc:
  image: tuttas/openplc_v3
  container_name: openplc
  ports:
    - "8080:8080"
    - "502:502"
  networks:
    ot-lab: { ipv4_address: 172.28.0.10 }
```

- Web: `http://localhost:8080` → usuario `openplc` / contraseña `openplc`.
- Programs → sube un `.st` (hay ejemplos en el repo de OpenPLC) → Launch program. En **Settings** confirma que Modbus Server está habilitado en el puerto 502.
- Para que OpenPLC actúe como **maestro** y lea el esclavo simulado: Slave Devices → Add → Generic Modbus TCP, IP `172.28.0.30`, puerto `5020`, y mapea holding registers a `%IW100`. Así generas tráfico Modbus entre contenedores sin tocar nada del host.

Nota: la imagen `tuttas/openplc_v3` es comunitaria (amd64). Si el contenedor no arranca o estás en ARM, clona [https://github.com/thiagoralves/OpenPLC\\_v3](https://github.com/thiagoralves/OpenPLC_v3) y construye tu propia imagen con el Dockerfile del repo.

### 3.3 Cliente Modbus desde un contenedor (mbpoll / pymodbus)

Sin instalar nada en el host:

```
# Leer 3 holding registers desde la dirección 40001 (offset 0) del esclavo simulado
docker run --rm --network ot-lab python:3.12-slim sh -c \
  "pip -q install pymodbus && python - <<'EOF'
  from pymodbus.client import ModbusTcpClient
  c = ModbusTcpClient('172.28.0.30', port=5020); c.connect()
  print(c.read_holding_registers(0, count=3, slave=1).registers)
  c.write_register(1, 42, slave=1)
  print(c.read_holding_registers(0, count=3, slave=1).registers)
  c.close()
  EOF"
```

Deberías ver [1234, 25, 72] y luego [1234, 42, 72].

Alternativa con mbpoll (Debian): `docker run --rm --network ot-lab debian:bookworm sh -c "apt-get -qq update && apt-get -qq install -y mbpoll >/dev/null && mbpoll -m tcp -a 1 -r 1 -c 3 -p 5020 172.28.0.30 -1"`.

## 4. Paso 3 — OPC UA

### 4.1 Servidor simulado: OPC PLC de Microsoft

Genera nodos con valores aleatorios, rampas, anomalías, etc. Ideal para probar clientes y seguridad OPC UA.

```
opc-plc:
  image: mcr.microsoft.com/iotedge/opc-plc:latest
  container_name: opc-plc
  command: >
    --pn=50000 --autoaccept --sph --sn=5 --sr=10 --st=uint
    --fn=5 --fr=1 --ft=uint --gn=5 --ut --dca
  ports:
    - "50000:50000"
  networks:
    ot-lab: { ipv4_address: 172.28.0.20 }
```

- Endpoint: `opc.tcp://localhost:50000`
- `--autoaccept` acepta automáticamente certificados de clientes (solo para laboratorio).
- `--ut` habilita autenticación anónima y por usuario; `--dca` deshabilita el chequeo de dominio del certificado. Quita estas opciones cuando quieras practicar certificados.

### 4.2 Servidor OPC UA propio en Python (asyncua)

opcua/server.py:

```
import asyncio, math
from asyncua import Server

async def main():
    srv = Server()
    await srv.init()
    srv.set_endpoint("opc.tcp://0.0.0.0:4840/lab/")
    srv.set_server_name("Lab OPC UA Python")
    ns = await srv.register_namespace("http://lab.local")
    obj = await srv.nodes.objects.add_object(ns, "Tanque1")
    nivel = await obj.add_variable(ns, "Nivel", 0.0)
    temp = await obj.add_variable(ns, "Temperatura", 20.0)
    valv = await obj.add_variable(ns, "Valvula", False)
    await valv.set_writable()
    t = 0
    async with srv:
        while True:
            t += 1
            await nivel.write_value(50 + 30 * math.sin(t / 10))
            await temp.write_value(20 + 5 * math.cos(t / 15))
            await asyncio.sleep(1)

asyncio.run(main())
```

opcua/Dockerfile:

```
FROM python:3.12-slim
RUN pip install --no-cache-dir asyncua
COPY server.py /app/server.py
CMD ["python", "/app/server.py"]
```

Servicio:

```
opcua-py:
  build: ./opcua
  container_name: opcua-py
  ports:
    - "4840:4840"
  networks:
    ot-lab: { ipv4_address: 172.28.0.21 }
```

### 4.3 Cliente: UaExpert (host)

1. Add Server → Custom Discovery → `opc.tcp://localhost:50000` y `opc.tcp://localhost:4840/lab/`.
2. Conéctate primero con **None / None** (sin seguridad) y captura el tráfico: verás los mensajes OPC UA en claro (Hello, OpenSecureChannel, CreateSession, Read, Publish).
3. Reconéctate con **Basic256Sha256 / Sign & Encrypt**, acepta el certificado del servidor y vuelve a capturar: ahora solo verás el handshake y luego payload cifrado. Ese contraste es el ejercicio clave de red en OPC UA.

Cliente en contenedor, por si no quieres UaExpert:

```
docker run --rm --network ot-lab python:3.12-slim sh -c \
  "pip -q install asyncua && python - <<'EOF'
import asyncio
from asyncua import Client
async def main():
    async with Client('opc.tcp://172.28.0.21:4840/lab/') as c:
        n = await c.nodes.root.get_child(['0:Objects', '2:Tanque1', '2:Nivel'])
        print('Nivel =', await n.read_value())
asyncio.run(main())
EOF"
```

## 5. Paso 4 — SCADA

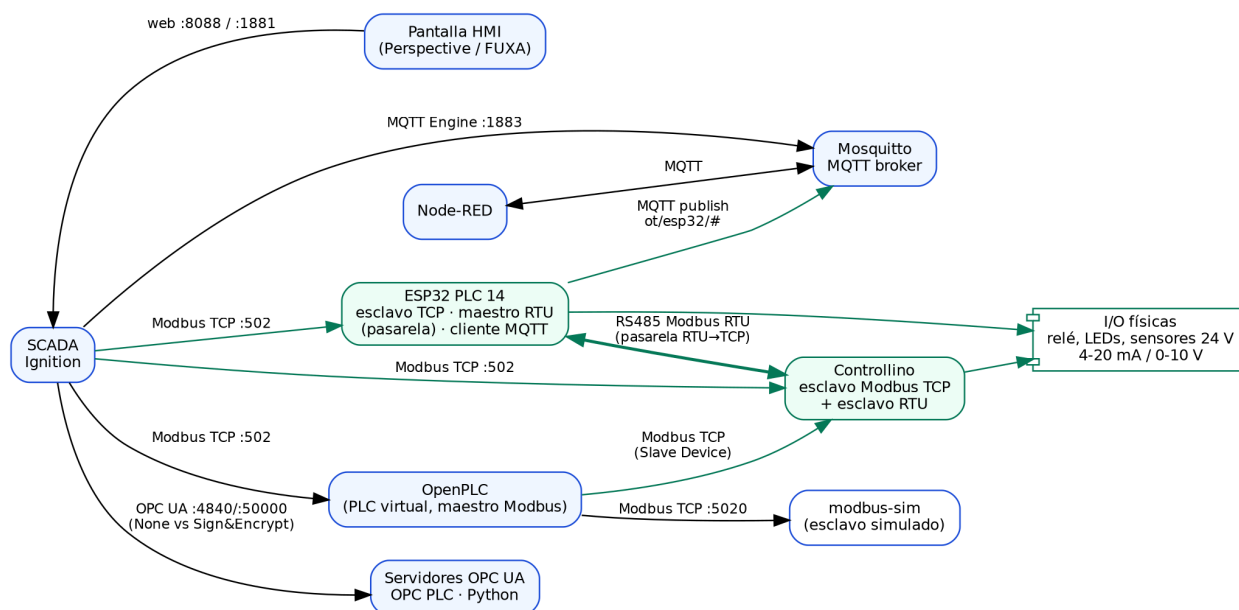


Figura 3. Diagrama de bloques: flujo de datos y protocolos entre los componentes

### 5.1 Ignition (Inductive Automation)

Imagen oficial, trial ilimitado en bloques de 2 horas reiniciables.

```
ignition:
  image: inductiveautomation/ignition:latest
  container_name: ignition
  environment:
    ACCEPT_IGNORE_EULA: "Y"
    GATEWAY_ADMIN_PASSWORD: "lab1234!"
    IGNITION_EDITION: "standard"
  volumes:
    - ./ignition-data:/usr/local/bin/ignition/data
  ports:
    - "8088:8088"
  networks:
    ot-lab: { ipv4_address: 172.28.0.50 }
```

- Web: `http://localhost:8088` → admin / lab1234!.
- **Config → OPC UA → Connections → Create new:** endpoint `opc.tcp://172.28.0.20:50000` (OPC PLC) y `opc.tcp://172.28.0.21:4840/lab/` (Python). Ignition tiene su propio cliente OPC UA y también expone un servidor OPC UA en el 62541.
- **Config → Modbus → Modbus TCP:** IP 172.28.0.10 puerto 502 (OpenPLC) e IP 172.28.0.30 puerto 5020 (simulador).
- Descarga el **Designer Launcher** desde la web del gateway, crea un proyecto Perspective y arrastra tags a una pantalla.

## 5.2 FUXA (open source, ligero)

```
fuxa:
  image: frangoteam/fuxa:latest
  container_name: fuxa
  volumes:
    - ./fuxa-data:/usr/src/app/FUXA/server/_appdata
  ports:
    - "1881:1881"
  networks:
    ot-lab: { ipv4_address: 172.28.0.51 }
```

Web: <http://localhost:1881> → Connections → añade Modbus TCP (172.28.0.30:5020) y OPC UA (opc.tcp://172.28.0.21:4840/lab/).

## 5.3 Node-RED (pasarela / pruebas rápidas)

```
node-red:
  image: nodered/node-red:latest
  container_name: node-red
  volumes:
    - ./nodered-data:/data
  ports:
    - "1880:1880"
  networks:
    ot-lab: { ipv4_address: 172.28.0.52 }
```

En <http://localhost:1880> → Manage palette → instala `node-red-contrib-modbus` y `node-red-contrib-opcua`. Útil para montar flujos que leen Modbus y publican OPC UA (o MQTT), y para generar tráfico continuo que capturar.

## 6. Paso 5 — Captura y análisis de tráfico

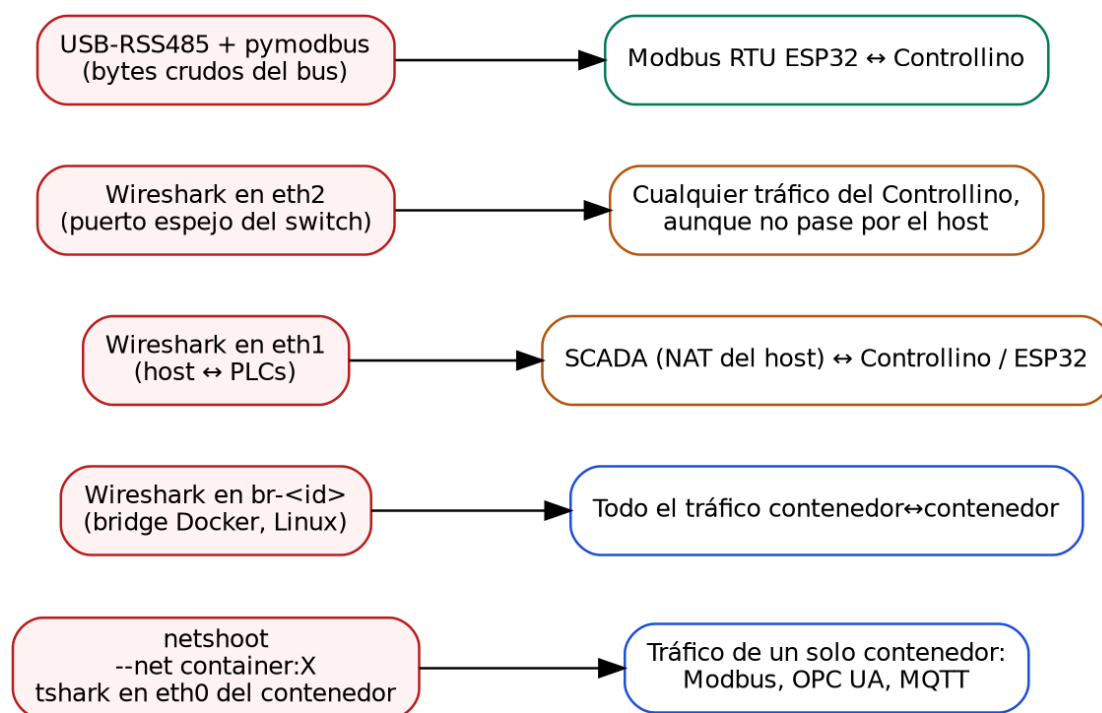


Figura 5. Dónde capturar tráfico y qué se ve en cada punto

### 6.1 Capturar dentro de la red Docker con netshoot

`nicolaka/netshoot` trae `tshark`, `nmap`, `tcpdump`, etc. La forma más limpia es **entrar al namespace de red de un contenedor** y capturar en su `eth0`:

```
# Ver 30 s de tráfico Modbus que llega a OpenPLC
docker run --rm --net container:openplc nicolaka/netshoot \
  tshark -i eth0 -a duration:30 -f "tcp port 502" -Y modbus

# Guardar a pcap para abrir en Wireshark
docker run --rm --net container:opc-plc -v $PWD/captures:/cap nicolaka/netshoot \
  tcpdump -i eth0 -w /cap/opcu.pcap "tcp port 50000"
```

Luego abre `captures/opcu.pcap` en Wireshark del host.

### 6.2 Capturar desde el host (Linux)

```
docker network inspect ot-lab -f '{{.Id}}' | cut -c1-12 # → br-<id>
sudo wireshark -i br-<id> -k -f "tcp port 502 or tcp port 5020 or tcp port 50000 or tcp port 4840"
```

## 6.3 Filtros útiles en Wireshark

| Filtro                             | Qué muestra                                                 |
|------------------------------------|-------------------------------------------------------------|
| mbtcp                              | Modbus TCP completo                                         |
| modbus.func_code == 16             | Escrituras de múltiples registros                           |
| modbus.exception_code              | Respuestas de error                                         |
| opcua                              | OPC UA (decodifica en claro solo si la sesión es None/None) |
| opcua.servicenodeid.numeric == 631 | Publish requests                                            |
| s7comm                             | Tráfico S7 (contra Conpot)                                  |
| enip \\ \\  cip                    | EtherNet/IP y CIP                                           |
| pn_io \\ \\  pn_dcp                | PROFINET (necesitas hardware o PLCSIM Advanced)             |

Wireshark → Analyze → **Conversations** y Statistics → **I/O Graph** te dan la periodicidad de polling: un SCADA sondeando cada segundo es una firma de red que conviene reconocer.

## 6.4 Escaneo y reconocimiento (contra tu propio lab)

```
docker run --rm --network ot-lab nicolaka/netshoot \
  nmap -sT -p 502,5020,102,4840,50000,8080,1881,8088 172.28.0.0/24

# Scripts NSE específicos de ICS
docker run --rm --network ot-lab nicolaka/netshoot \
  nmap -p 502 --script modbus-discover 172.28.0.10 172.28.0.30
```

## 7. Paso 6 — Honeypot ICS: Conpot

---

Simula un PLC S7-300 con S7comm, Modbus, SNMP y HTTP. Sirve para ver cómo se ve un dispositivo "real" ante un escáner y para practicar detección.

```
conpot:
  image: honeynet/conpot:latest
  container_name: conpot
  command: conpot --template default --logfile /var/log/conpot/conpot.log
  ports:
    - "10502:502"
    - "10102:102"
    - "10080:80"
    - "10161:161/udp"
  networks:
    ot-lab: { ipv4_address: 172.28.0.90 }
```

Prueba: `nmap -p 502,102 --script modbus-discover,s7-info 172.28.0.90` desde netshoot y revisa `docker logs conpot`: verás el escaneo registrado como lo haría un IDS.

## 8. Paso 7 — Integrar los PLCs físicos: Controllino y ESP32 PLC 14

Aquí es donde el laboratorio deja de ser simulado. Ambos dispositivos se programan con Arduino IDE y hablan Modbus, así que encajan como **dispositivos de nivel 1/0** que el SCADA y el OpenPLC del Docker pueden leer.

### 8.1 Qué tienes en cada equipo

|                      | Controllino                                                           | ESP32 PLC 14 (Industrial Shields)                                                  |
|----------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| MCU                  | ATmega2560 (MAXI/MEGA) o ATmega328 (MINI)                             | ESP32 dual-core 240 MHz                                                            |
| Ethernet             | Sí en MAXI/MEGA/MAXI Automation (W5100, 10/100). <b>MINI no tiene</b> | Sí (RJ45)                                                                          |
| Inalámbrico          | No                                                                    | WiFi + BLE                                                                         |
| Serie industrial     | RS485 en MAXI/MEGA                                                    | RS485 half-duplex                                                                  |
| Otros                | RTC, I/O 24 V                                                         | I2C para expansión, RTC, I/O 24 V, 1 relé, entradas 0-10 V / 4-20 mA               |
| Rol típico en el lab | Esclavo Modbus TCP "clásico", cableado, determinista                  | Esclavo Modbus TCP + pasarela RTU→TCP + publicador MQTT; comparar Ethernet vs WiFi |

Si tu Controllino es **MINI** (sin Ethernet ni RS485), conéctalo por USB y úsalo como esclavo Modbus RTU sobre serial: el host o el ESP32 hacen de pasarela. El resto de la guía asume MAXI o MEGA.

## 8.2 Topología física y direccionamiento

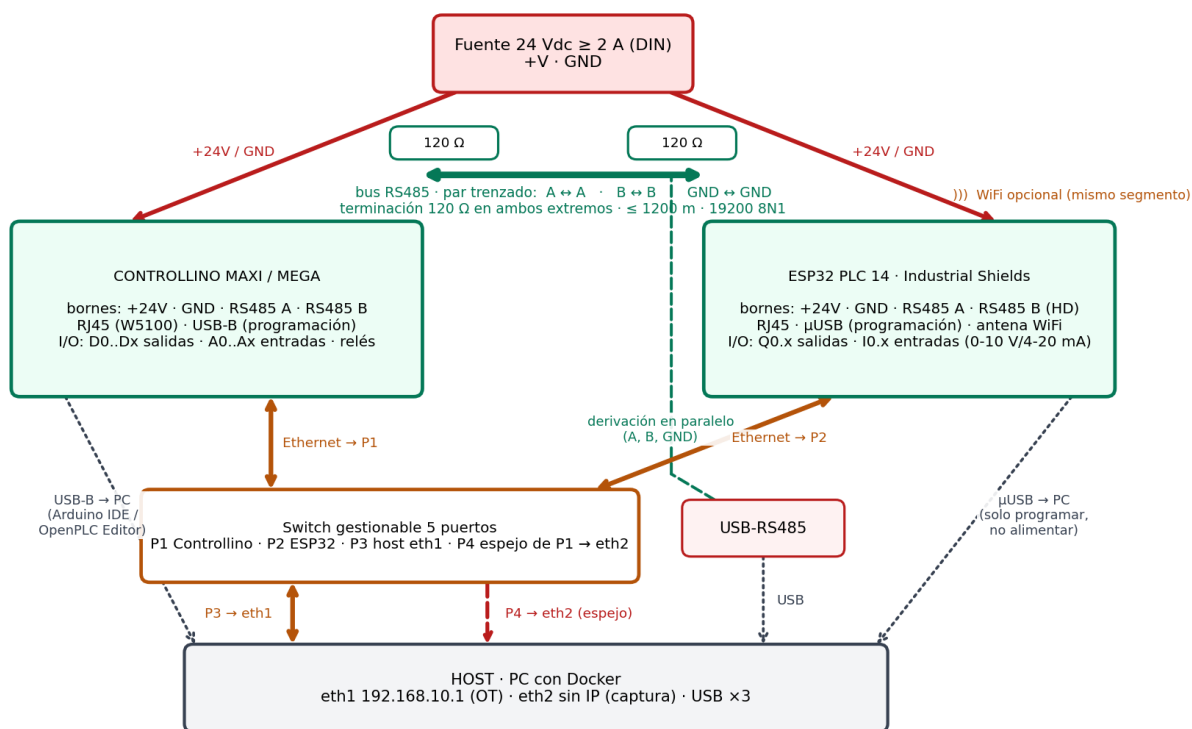


Figura 4. Conexiones físicas entre equipos: alimentación, Ethernet, bus RS485 y USB de programación

1. Conecta la segunda NIC del host (eth1, USB-Ethernet), el Controllino y el ESP32 a un switch.
2. Asigna IPs estáticas: host 192.168.10.1/24, Controllino .11, ESP32 .12. Sin gateway por defecto en los PLCs: no tienen por qué salir a ningún sitio.
3. En Linux, configura la NIC del host con los comandos que siguen a esta lista y comprueba que ambos PLCs responden a ping.
4. Si el switch es gestionable, configura **port mirroring** del puerto del Controllino hacia un puerto donde tengas una tercera NIC (o hacia eth1 si aceptas mezclar). Así Wireshark ve el tráfico entre el SCADA y el PLC aunque no pase por el host: es exactamente como se despliega un sensor de red (Nozomi, Claroty, Zeek) en una planta.

```
sudo ip addr add 192.168.10.1/24 dev eth1 && sudo ip link set eth1 up
ping -c 3 192.168.10.11 && ping -c 3 192.168.10.12
```

## 8.3 Cómo llegan los contenedores a los PLCs

Tienes dos opciones; empieza con la A y pasa a la B cuando quieras que los contenedores sean "hosts de verdad" en la red OT.

**A) Bridge + NAT (por defecto, funciona en cualquier SO).** Los contenedores salen por NAT del host, así que Ignition/FUXA/OpenPLC alcanzan 192.168.10.11:502 sin configurar nada. Limitación: los PLCs no pueden iniciar conexiones a un contenedor salvo a través de un puerto publicado en el host (p. ej. el ESP32 publica MQTT a 192.168.10.1:1883, que es el puerto publicado de mosquitto).

**B) macvlan (solo Linux nativo).** Cada contenedor recibe una IP real en 192.168.10.0/24 y una MAC propia; en Wireshark aparecen como equipos distintos y puedes hacer nmap "desde fuera".

```

networks:
  ot-phys:
    driver: macvlan
    driver_opts:
      parent: eth1
    ipam:
      config:
        - subnet: 192.168.10.0/24
          ip_range: 192.168.10.128/25  # rango reservado a contenedores
          gateway: 192.168.10.1

```

Añade a `ignition`, `fluxa`, `node-red` y `openplc` una segunda red: `ot-phys: { ipv4_address: 192.168.10.150 }` (y sucesivas). Detalle importante de macvlan: el host **no** puede hablar con sus propios contenedores macvlan por `eth1`; crea una interfaz puente:

```

sudo ip link add ot-shim link eth1 type macvlan mode bridge
sudo ip addr add 192.168.10.2/32 dev ot-shim && sudo ip link set ot-shim up
sudo ip route add 192.168.10.128/25 dev ot-shim

```

## 8.4 Firmware: opción rápida con OpenPLC Editor

OpenPLC Editor puede compilar un programa IEC 61131-3 y subirlo directamente a placas Arduino, **incluidos Controllino MAXI/MEGA/MINI y ESP32**, dejando la placa como esclavo Modbus (TCP y/o RTU) con los I/O mapeados a `%IX`, `%QX`, `%IW`, `%QW`.

1. OpenPLC Editor → nuevo proyecto → escribe un ladder sencillo (p. ej. `%QX0.0 := %IX0.0`).
2. Menú **Transfer program to PLC** (icono de flecha) → Board: *Controllino MAXI / ESP32* → pestaña **Modbus**: habilita TCP, pon IP `192.168.10.11` (Controllino) o `.12` (ESP32), puerto 502; en el ESP32 puedes elegir Ethernet o WiFi (SSID/clave).
3. Compila y sube. El dispositivo ya responde Modbus TCP.
4. En el OpenPLC del Docker (<http://localhost:8080>) → **Slave Devices** → **Add** → Generic Modbus TCP, IP del PLC físico, y mapea sus registros: ahora el PLC virtual controla I/O reales. En Ignition, añade ambos como dispositivos Modbus TCP igual que hiciste con el simulador.

Para el ESP32 PLC 14, Industrial Shields documenta el mapa de pines de sus placas; comprueba en su web qué pines de Arduino corresponden a `I0.x/Q0.x` para usarlos desde OpenPLC.

## 8.5 Firmware: opción con Arduino IDE (control total)

Cuando quieras controlar tú el protocolo, usa las librerías del fabricante. En Arduino IDE instala el paquete de placas **industrialshields-esp32** (Gestor de tarjetas) y la librería **Tools40** de Industrial Shields (trae `ModbusTCPSlave`, `ModbusTCPMaster`, `ModbusRTUSlave`, `ModbusRTUMaster`). Para el Controllino instala las placas Controllino y la librería **ArduinoModbus** (con `ArduinoRS485`).

**ESP32 PLC 14 como esclavo Modbus TCP por Ethernet** (esqueleto; parte de *File* → *Examples* → *Tools40* para el modelo exacto):

```

#include <Ethernet.h>
#include <ModbusTCPSlave.h>

byte mac[] = {0xDE,0xAD,0xBE,0xEF,0x00,0x12};
IPAddress ip(192,168,10,12);
ModbusTCPSlave modbus(502);

bool    coils[4];
bool    dinputs[4];
uint16_t hregs[4];
uint16_t iregs[4];

void setup() {
  Ethernet.begin(mac, ip);
  modbus.begin();
  modbus.setCoils(coils, 4);
  modbus.setDiscreteInputs(dinputs, 4);
  modbus.setHoldingRegisters(hregs, 4);
  modbus.setInputRegisters(iregs, 4);
}

void loop() {
  dinputs[0] = digitalRead(I0_0);
  iregs[0] = analogRead(I0_5);           // entrada analógica
  digitalWrite(Q0_0, coils[0]);         // el SCADA escribe el coil 0 → sale por Q0.0
  modbus.update();
}

```

### Controllino MAXI como esclavo Modbus TCP (ArduinoModbus):

```

#include <Controllino.h>
#include <SPI.h>
#include <Ethernet.h>
#include <ArduinoRS485.h>
#include <ArduinoModbus.h>

byte mac[] = {0xDE,0xAD,0xBE,0xEF,0x00,0x11};
EthernetServer server(502);
ModbusTCPServer modbusTCP;

void setup() {
  Ethernet.begin(mac, IPAddress(192,168,10,11));
  server.begin();
  modbusTCP.begin();
  modbusTCP.configureCoils(0, 8);
  modbusTCP.configureDiscreteInputs(0, 8);
  modbusTCP.configureHoldingRegisters(0, 8);
  pinMode(CONTROLLINO_D0, OUTPUT);
  pinMode(CONTROLLINO_A0, INPUT);
}

void loop() {
  EthernetClient client = server.available();
  if (client) {
    modbusTCP.accept(client);
    while (client.connected()) {
      modbusTCP.poll();
      modbusTCP.discreteInputWrite(0, digitalRead(CONTROLLINO_A0));
      digitalWrite(CONTROLLINO_D0, modbusTCP.coilRead(0));
    }
  }
}

```

Con cualquiera de los dos firmwares, comprueba desde el host: `mbpoll -m tcp -a 1 -t 0 -r 1 -c 4 192.168.10.11` (coils) y escribe un coil con `mbpoll -m tcp -a 1 -t 0 -r 1 192.168.10.11 1`: debes oír el relé/ver el LED de salida.

## 8.6 Modbus RTU sobre RS485 entre los dos PLCs (y el ESP32 como pasarela)

Cablea A↔A, B↔B (y GND) entre el RS485 del Controllino y el del ESP32. Controllino como **esclavo RTU** (ArduinoModbus ModbusRTU Server, 19200 8N1, ID 1); ESP32 como **maestro RTU** (ModbusRTUMaster de Tools40) que lee los registros del Controllino y los copia a sus propios holding registers Modbus TCP. Resultado: desde Ignition lees el Controllino "a través" del ESP32, que actúa de **pasarela RTU→TCP**, que es exactamente lo que hacen los gateways Moxa/Anybus en planta.

Para analizar el bus serie: conecta el adaptador USB-RS485 del host en paralelo al bus (A, B, GND) y captura los bytes crudos con pymodbus (ModbusSerialClient con framer='rtu') o con un script que vuelque serial.read() en hex; Wireshark no captura serial directamente, pero puedes reinyectar el volcado o usar socat para exponerlo por TCP y decodificarlo como mbrtu con Decode As.

## 8.7 MQTT desde el ESP32 (IIoT, el otro protocolo que verás en planta)

Añade el broker al compose (ya está en la versión final): eclipse-mosquitto en 172.28.0.60:1883, publicado en el host. En el ESP32, con PubSubClient, publica cada segundo ot/esp32/nivel y suscríbete a ot/esp32/cmd. Node-RED e Ignition (módulo MQTT Engine) lo consumen. Ejercicio de red: MQTT viaja en claro sobre TCP 1883; captura, lee el payload, y luego activa TLS en mosquitto (8883) para ver la diferencia. Cambia el ESP32 de Ethernet a WiFi y compara jitter en Wireshark (Statistics → I/O Graph con el filtro mqtt).

## 9. Lo que no cabe bien en Docker: Ethernet industrial de tiempo real

---

PROFINET, EtherNet/IP con I/O implícito y EtherCAT trabajan en capa 2 (tramas Ethernet propias, multicast, tiempos de ciclo de ms). El bridge de Docker es capa 3 y en Windows/macOS ni siquiera ves la NIC real. Opciones:

| Protocolo              | Opción recomendada                                                                           | Notas                                                                                                                                                    |
|------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| PROFINET               | Siemens TIA Portal + <b>PLCSIM Advanced</b> (Windows, trial)                                 | Expone un controlador PROFINET virtual sobre la NIC del PC; captura con Wireshark filtro <code>pn_dcp</code> para ver el descubrimiento de dispositivos. |
| PROFINET (dispositivo) | <b>p-net</b> (RT-Labs) en Raspberry Pi o VM Linux con NIC dedicada                           | Compila el sample app; puede correr en contenedor con <code>network_mode: host</code> y <code>cap_add: NET_ADMIN, NET_RAW</code> solo en Linux nativo.   |
| EtherNet/IP            | <b>OpENer</b> (adaptador open source) o el módulo EtherNet/IP de Ignition contra un PLC real | OpENer sí funciona en contenedor Linux con <code>network_mode: host</code> .                                                                             |
| EtherCAT               | <b>SOEM</b> + un esclavo real barato (EL1008 usado)                                          | No hay simulador serio sin hardware.                                                                                                                     |

Con el Controllino y el ESP32 ya cubres Modbus TCP/RTU y MQTT sobre hardware real. Para PROFINET/EtherNet/IP reales, un S7-1200 o un Micro820 de segunda mano completan el laboratorio.

## 10. Ejercicios de red sugeridos (en orden)

---

1. **Anatomía Modbus:** captura una lectura y una escritura; identifica MBAP header (transaction ID, unit ID), function code y datos. Provoca una excepción leyendo un registro que no existe.
2. **Polling:** configura Ignition para leer OpenPLC cada 500 ms y mide en I/O Graph paquetes/segundo. Cambia a 100 ms y compara.
3. **OPC UA en claro vs cifrado:** dos capturas de UaExpert (None vs Sign&Encrypt); explica qué sigue siendo visible (endpoints, certificados) y qué no.
4. **Reconocimiento:** nmap sobre la red y clasifica cada host por función solo a partir de puertos y banners.
5. **Segmentación:** crea una segunda red `ot-dmz` en compose, mueve Ignition allí y deja Node-RED conectado a ambas como "pasarela". Verifica con nmap que Ignition ya no ve a OpenPLC directamente.
6. **Detección:** lanza un escaneo y una escritura Modbus no autorizada contra Conpot; correlaciona el log del honeypot con la captura.
7. **Manipulación:** desde pymodbus escribe un coil de OpenPLC mientras Ignition lo muestra en pantalla; observa el efecto y lo que aparece en la captura.

Con los PLCs físicos:

1. **Simulado vs real:** Ignition sondea a la vez `modbus-sim`, OpenPLC (Docker) y el Controllino. En Wireshark compara el *delta time* petición→respuesta de cada uno; el Controllino (W5100, AVR a 16 MHz) será visiblemente más lento. Discute qué implica para el tiempo de sondeo del SCADA.
2. **Port mirroring:** captura el tráfico Ignition ↔ Controllino desde una NIC en el puerto espejo del switch, sin tocar el host. Es la posición de un sensor pasivo de red OT.
3. **Ethernet vs WiFi:** mismo firmware en el ESP32 por cable y por WiFi; mide latencia, jitter y retransmisiones (`tcp.analysis.retransmission`). Añade interferencia (microondas, descargas en la misma WiFi) y repite.
4. **Pasarela RTU→TCP:** implementa §8.6 y observa que una sola petición Modbus TCP al ESP32 desencadena tráfico RS485 hacia el Controllino; mide el tiempo extra que añade la pasarela.
5. **Segmentación real:** con `nftables` en el host permite solo TCP 502 desde la IP de Ignition hacia `192.168.10.0/24`. Verifica con nmap desde otro contenedor que ya no alcanza los PLCs; luego intenta la escritura de coil desde un equipo de tu red doméstica y comprueba que falla.
6. **Escritura no autorizada con efecto físico:** desde un tercer host escribe el coil del relé del Controllino/ESP32 y míralo conmutar. Diseña a partir de ahí una regla de detección (Zeek o Suricata con reglas Modbus) que alerte de escrituras desde IPs que no sean el SCADA.
7. **MQTT en claro vs TLS:** §8.7; compara capturas en 1883 y 8883.

## 11. Problemas frecuentes

---

- **Puerto 502 ocupado o requiere privilegios:** en Linux mapea a 1502:502 si no quieres correr como root.
- **UaExpert no conecta a opc-plc:** verifica que el certificado del servidor está en Trusted; con `--autoaccept` el servidor acepta el tuyo, pero tú debes aceptar el suyo.
- **Ignition no ve los contenedores:** usa las IPs de la red `ot-lab` (172.28.0.x), no `localhost`, porque Ignition también corre dentro de Docker.
- **Imagen amd64 en ARM (Mac M1/M2, Raspberry):** `exec format error`. Añade `platform: linux/amd64` al servicio o construye la imagen localmente.
- **Wireshark no decodifica OPC UA:** Analyze → Decode As → puerto 4840/50000 → OpcUa.
- **El Controllino responde a ping pero no a Modbus:** el W5100 solo admite 4 sockets simultáneos; cierra clientes (UaExpert no, mbpoll/Ignition/FUXA sí cuentan). Ignition además abre varias conexiones por dispositivo: limita a 1 en la configuración del dispositivo.
- **El ESP32 se reinicia al recibir tráfico:** casi siempre es alimentación insuficiente por USB. Aliméntalo a 24 V.
- **macvlan no funciona:** solo en Linux nativo con NIC física (no en Docker Desktop, no en WiFi del host, que suele filtrar MACs ajenas). Usa la opción A (NAT).
- **RS485 con datos corruptos:** revisa polaridad A/B, GND común, y activa la resistencia de terminación de 120  $\Omega$  en los extremos (el ESP32 PLC 14 y el Controllino tienen jumper/switch para ello).



## 12. `docker-compose.yml` completo

---

```

name: ot-lab

networks:
  ot-lab:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/24

services:
  modbus-sim:
    image: oitc/modbus-server:latest
    container_name: modbus-sim
    command: -f /server_config.json
    volumes:
      - ./modbus/server.json:/server_config.json:ro
    ports: ["5020:5020"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.30 }

  openplc:
    image: tuttas/openplc_v3
    container_name: openplc
    ports: ["8080:8080", "502:502"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.10 }

  opc-plc:
    image: mcr.microsoft.com/iotedge/opc-plc:latest
    container_name: opc-plc
    command: >
      --pn=50000 --autoaccept --sph --sn=5 --sr=10 --st=uint
      --fn=5 --fr=1 --ft=uint --gn=5 --ut --dca
    ports: ["50000:50000"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.20 }

  opcua-py:
    build: ./opcua
    container_name: opcua-py
    ports: ["4840:4840"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.21 }

  ignition:
    image: inductiveautomation/ignition:latest
    container_name: ignition
    environment:
      ACCEPT_IGNITION_EULA: "Y"
      GATEWAY_ADMIN_PASSWORD: "lab1234!"
      IGNITION_EDITION: "standard"
    volumes:
      - ./ignition-data:/usr/local/bin/ignition/data
    ports: ["8088:8088"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.50 }

  fuxa:
    image: frangoteam/fuxa:latest
    container_name: fuxa
    volumes:
      - ./fuxa-data:/usr/src/app/FUXA/server/_appdata
    ports: ["1881:1881"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.51 }

  node-red:
    image: nodered/node-red:latest
    container_name: node-red
    volumes:
      - ./nodered-data:/data
    ports: ["1880:1880"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.52 }

  conpot:
    image: honeynet/conpot:latest

```

```

container_name: conpot
command: conpot --template default --logfile /var/log/conpot/conpot.log
ports: ["10502:502", "10102:102", "10080:80", "10161:161/udp"]
networks:
  ot-lab: { ipv4_address: 172.28.0.90 }

mosquitto:
image: eclipse-mosquitto:2
container_name: mosquitto
volumes:
  - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
ports: ["1883:1883", "8883:8883"]
networks:
  ot-lab: { ipv4_address: 172.28.0.60 }

```

mosquitto/mosquitto.conf mínimo para el laboratorio (sin autenticación, solo 1883; añade el listener 8883 con certificados cuando llegues al ejercicio TLS):

```

listener 1883
allow_anonymous true

```

Arranque completo:

```

cd ~/ot-lab
docker compose up -d
docker compose ps

```

Servicios web: OpenPLC :8080 · Ignition :8088 · FUXA :1881 · Node-RED :1880 · Conpot :10080. MQTT en :1883.